# FindYourBook

ISPW PROJECT - University of Rome Tor Vergata.
AURORA CICCHETTA
2025-2026

# FindYourBook

## Project Documentation

### Table of contents

# 1. Software Requirement Specification (SRS)

## 1.1 Introduction

### 1.1.1 Aim of the Document

The purpose of this document is to present the fundamental and specific requirements of the "FindYourBook" system, a project developed for the Software Engineering course, Academic Year 2025-2026. The document outlines the system's objectives, core functionalities, and architectural design choices to guide and document the development process.

### 1.1.2 Overview of the Defined System

The system is a Java-based software platform structured according to the MVC architecture, utilizing Maven for dependency management, and featuring graphical user interfaces implemented via JavaFX alongside support components and robust logging (AppLogger). The system supports the modern and efficient management of a literary catalog and personal user libraries, including the following key functionalities:

- Management of distinct user roles: Readers and Publishing Houses (*Publisher*).
- Browsing and searching within the official catalog of published books.
- Managing personal library states (*To Read*, *Reading*, *Read*) and rating books.
- Automatic tracking of reading progress and automated background or event-driven checks on reading duration.
- Email notifications for registration confirmation, book publications, reading goals reached, and inactivity reminders (via SendGrid API integration).

### 1.1.3 Hardware and Software Requirements

- Software requirements:
  - Java 21+ (compatible with OpenJFX 23+)
  - Maven 3.8+
  - MySQL Server 8.0+ (or in-memory/JSON fallback persistence for demo mode)
  - JavaFX 23+
  - IDE: IntelliJ IDEA (recommended) or Eclipse
- Hardware requirements:
  - CPU: Dual-Core 2.0 GHz
  - RAM: 4 GB (2 GB minimum)
  - Disk Space: 1 GB
  - Operating System: Windows, MacOS, Linux (compatible with Java 21+)

### 1.1.4 Related Systems - Pros and Cons

FindYourBook

- PRO:
    - Free of charge and completely open access for all users.
    - Targeted specifically at structured dual user roles, bridging readers and publishing houses directly.
    - Email notifications for registration confirmation, published book updates, reading goal achievements, and inactivity reminders (via SendGrid API integration).
- CONS:
    - Desktop-only software; no native mobile application available yet.
    - Inactivity checks and reading reminders are triggered locally during user login rather than synchronized with external cloud calendars.

Goodreads

- PRO:
    - Massive, global community of readers offering extensive reviews, ratings, and social interactions.
    - Cross-platform availability with robust mobile applications and web access.
    - Offers comprehensive personalized book recommendations based on user reading history and community ratings.
- CONS:
    - Heavy commercialization and advertising presence, lacking direct integration for independent publishing houses to manage catalogs and sales metrics.
    - Interface and recommendation algorithms can sometimes feel overwhelming or overly cluttered for users seeking a simple personal library manager.

## 1.2 User Stories

1. As a Reader, I want to save a new book into my personal library under the "To Read" section, so that I can track my future reading list directly from the application.
2. As a Publishing House, I want to add a new title to the platform's official catalog, so that it becomes immediately visible and accessible to all readers.
3. As a Reader, I want to select books by literary genre, so that I can choose my next reading material based on my current interests.

## 1.3 Functional Requirements

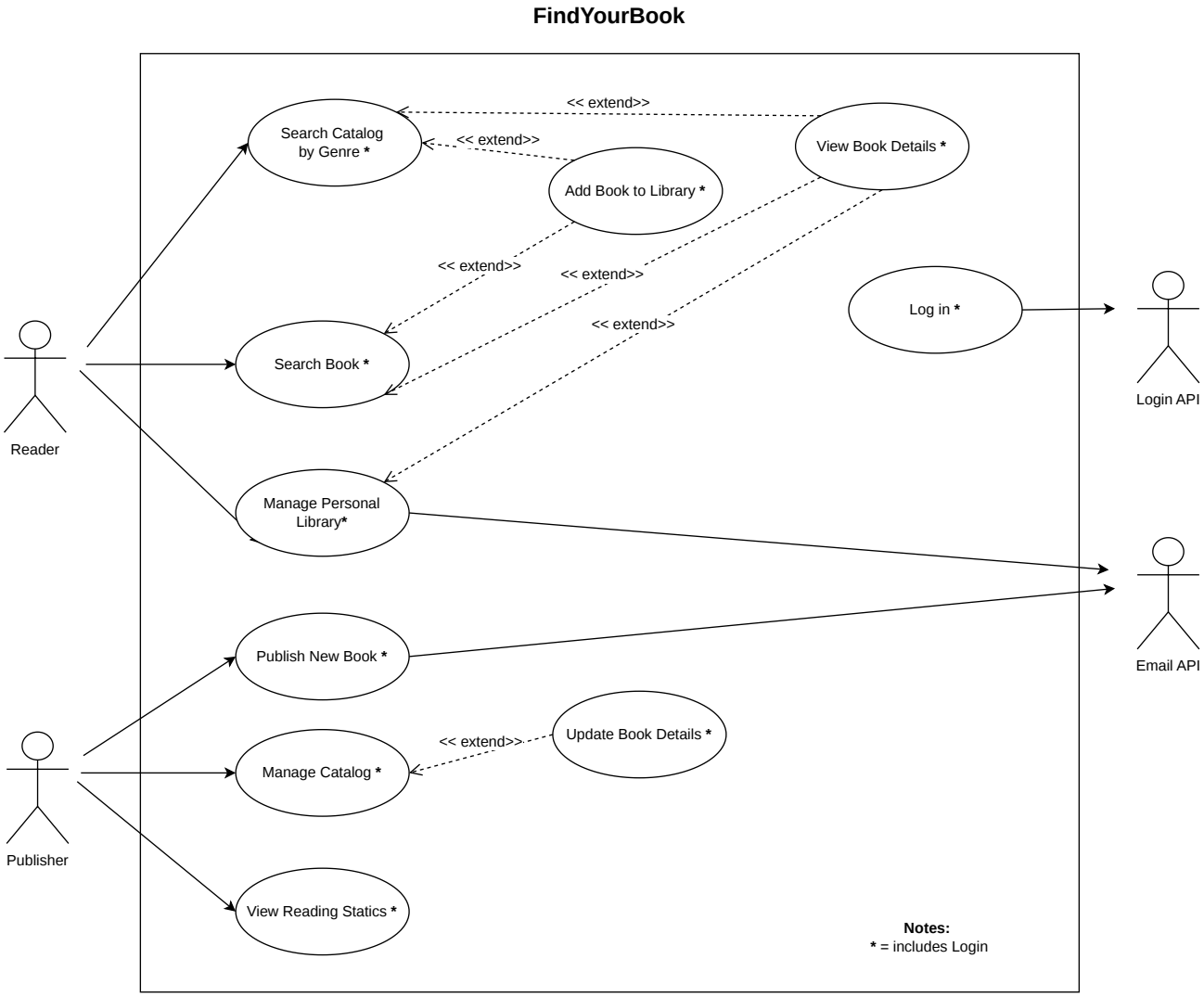
1. The system shall save to the Reader's personal library a book with the reading status selected by the Reader (To Read, Reading, or Read).
2. The system shall add a new book (title, author, genre, description, image URL) submitted by a Publishing House into the catalog.
3. The system shall display the catalog books matching the literary genre selected by the Reader.

Glossary

- Reader = A registered user of the platform whose primary role is to browse the catalog, manage personal library states, and track reading progress.
- Publishing House = A registered user of the platform whose primary role is to submit, update, and remove book titles within the official system catalog.
- Catalog = The collection of all books published in the system by every Publishing House, searchable and browsable by Readers. Each Publishing House can only view and manage the subset of the catalog made up of the titles it has submitted.
- Reading Status = The specific categorization state assigned to a book within a personal library, indicating whether it is marked as To Read, Reading, or Read.

# 1.4 Use Cases (Focus on Core Features)

## 1.4.1 Overview Diagram



### 1.4.2 Internal Steps

*Use case:* Manage Personal Library

*Main flow:*

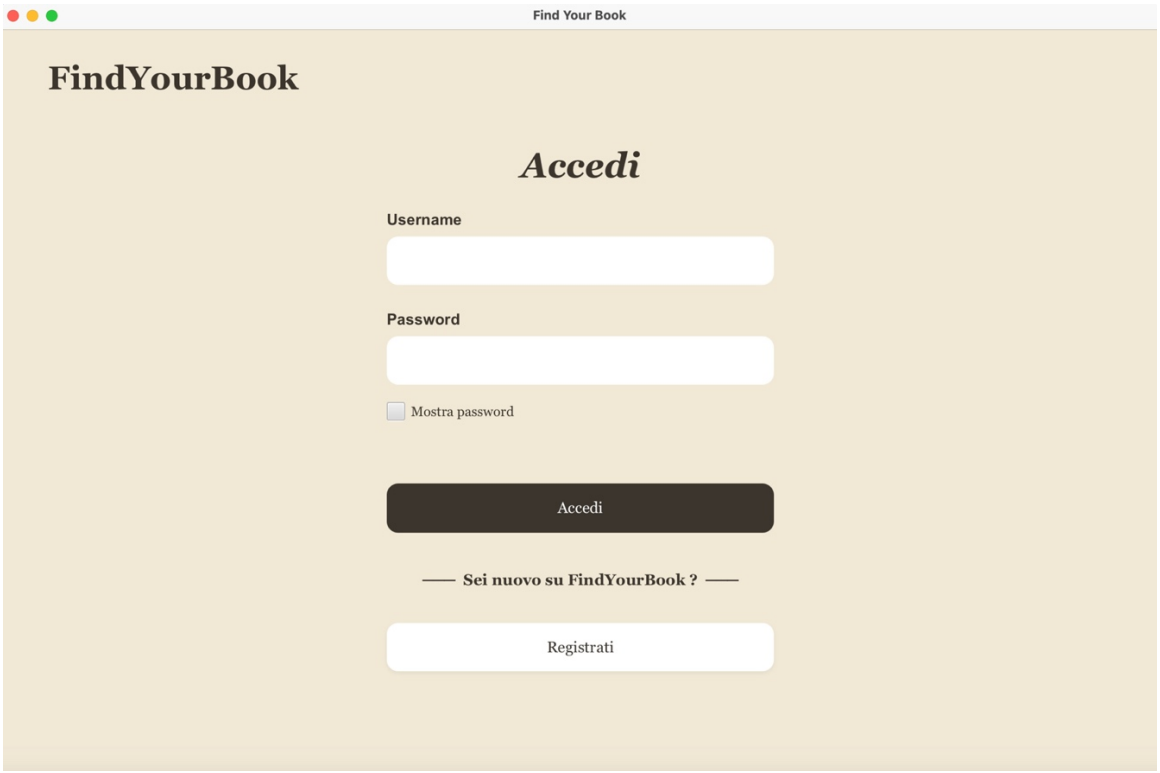
1. The reader accesses their personal account (via UC Login).
2. The reader selects the personal library function from the dashboard.
3. The system retrieves and displays the count of total read works associated with the reader's profile.
4. The system verifies whether any books currently in the Reading state have a start date older than 30 days.
5. The system presents the reading-status filters (To Read, Reading, Read).
6. The reader selects a specific reading status filter.
7. The system requests and loads the book collection filtered by the selected reading status.
8. The system displays the filtered book collection, including titles, authors, ratings, and current statuses.
9. The reader selects a specific book and performs one of the following actions:
   - The Reader updates the reading status (To Read, Reading, Read).
   - The Reader rates the book.
   - The Reader removes the book from the personal library.
10. The system updates the reader's personal library with the selected change.
11. The system refreshes the library view.

*Extensions :*

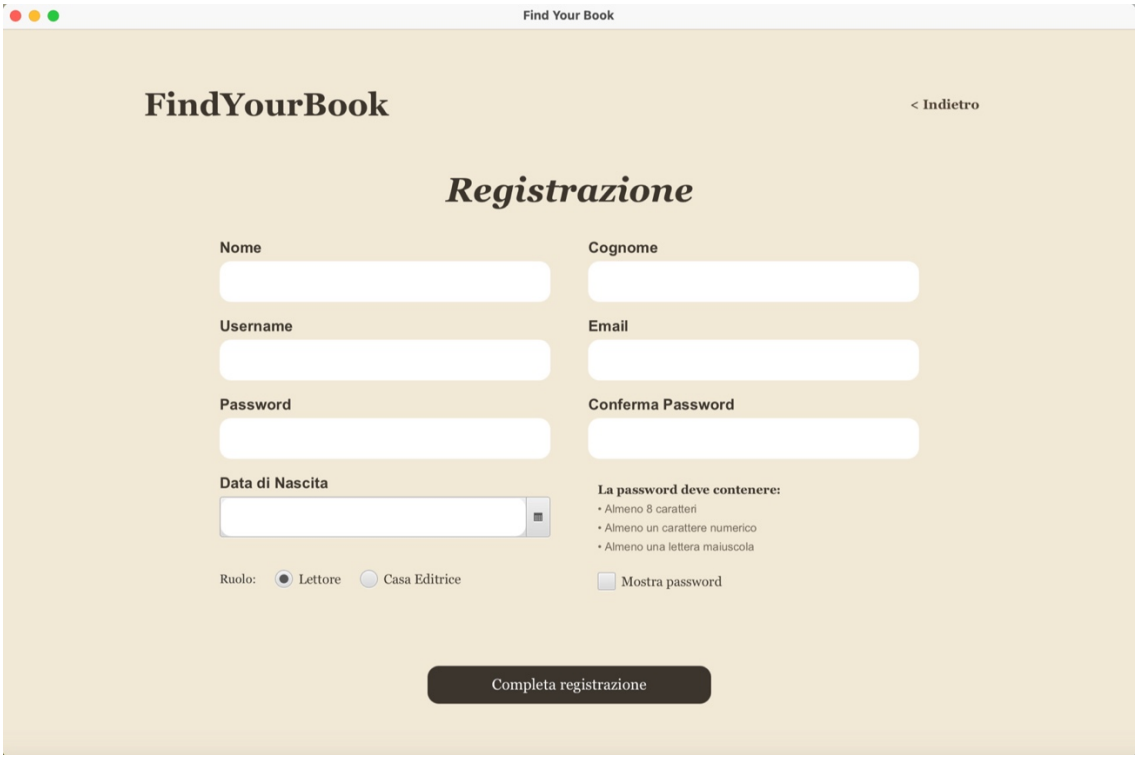
**4a.** Inactive reading detected: The system identifies books in the Reading state started over 30 days ago and invokes the Email API (SendGrid) to send a reading inactivity reminder to the Reader's email address, then continues the main flow from step 5.
**9a.** Completion goal reached: The Reader updates the book status specifically to Read; the system invokes the Email API to send a completion goal notification to the Reader's email address, then continues to step 10.
**9b.** Removal not confirmed: The Reader cancels the removal, or the 5-second confirmation countdown expires without confirmation; the system keeps the book in the library and terminates the use case.
**9c.** Optional detail view: The Reader, before choosing an action, may view the full details of the selected book (description, author, current status) via UC View Book Details; the system displays the book detail view, then the reader proceeds to choose one of the actions listed in step 9.

# 2. Storyboards

## Login

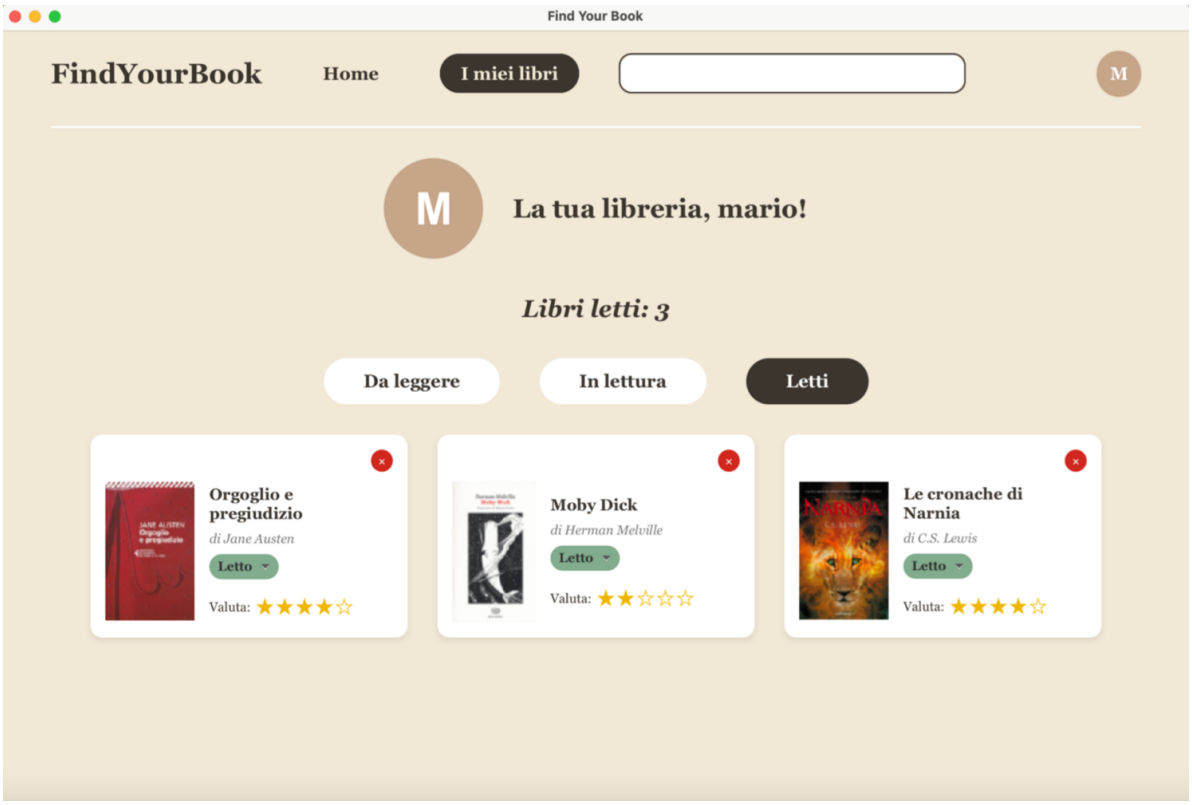



## Registration

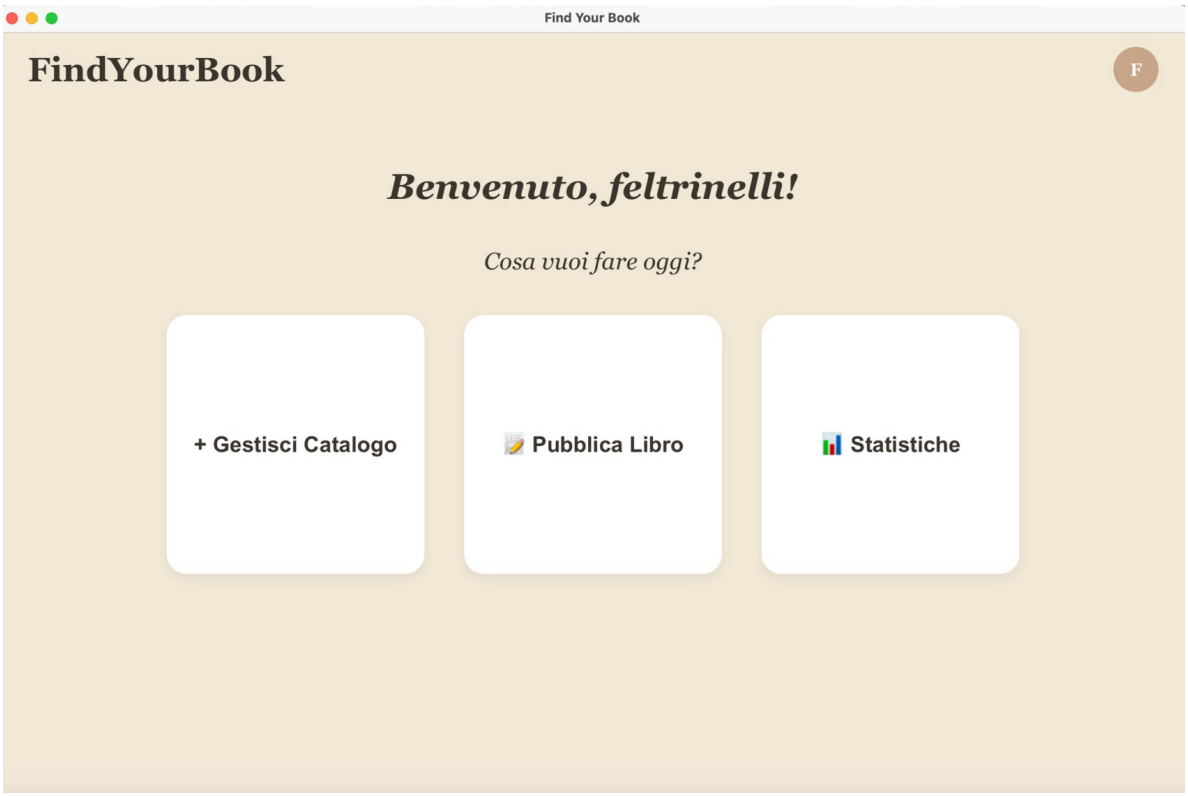


## Reader Dashboard

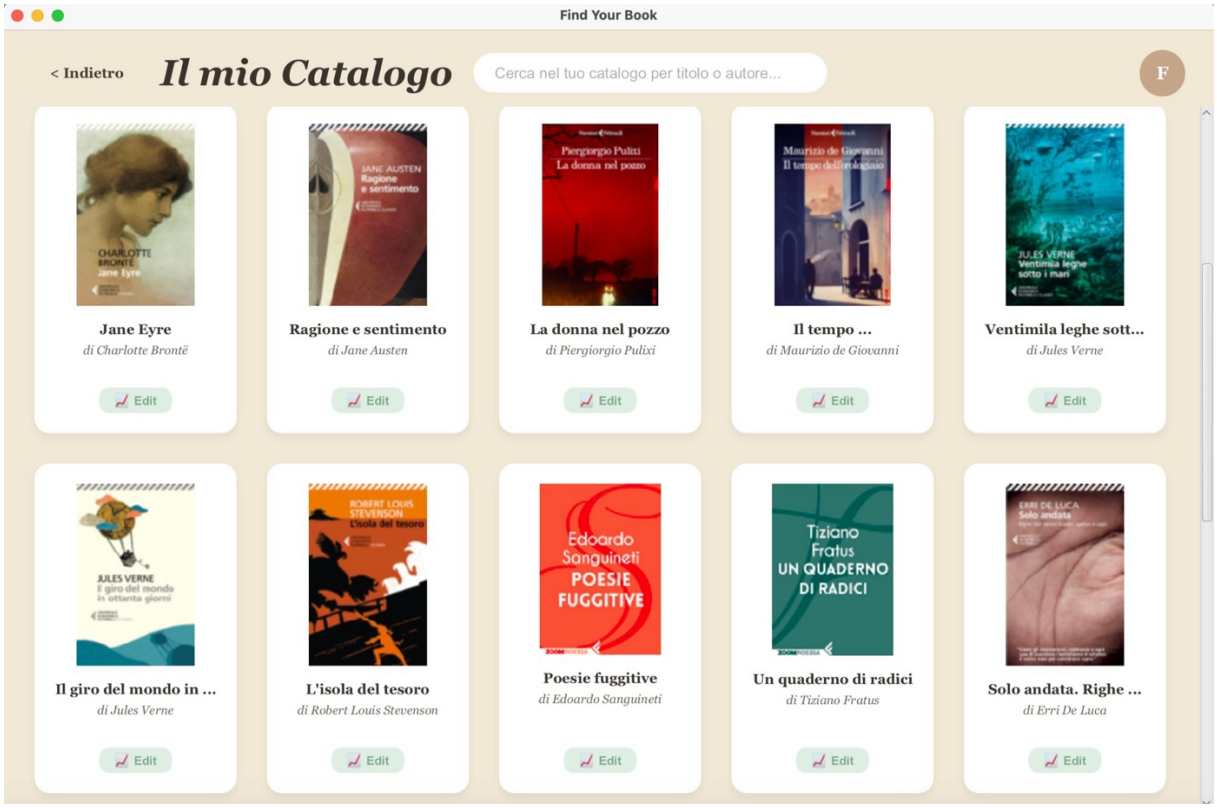

## Reader Library



## Publisher Dashboard



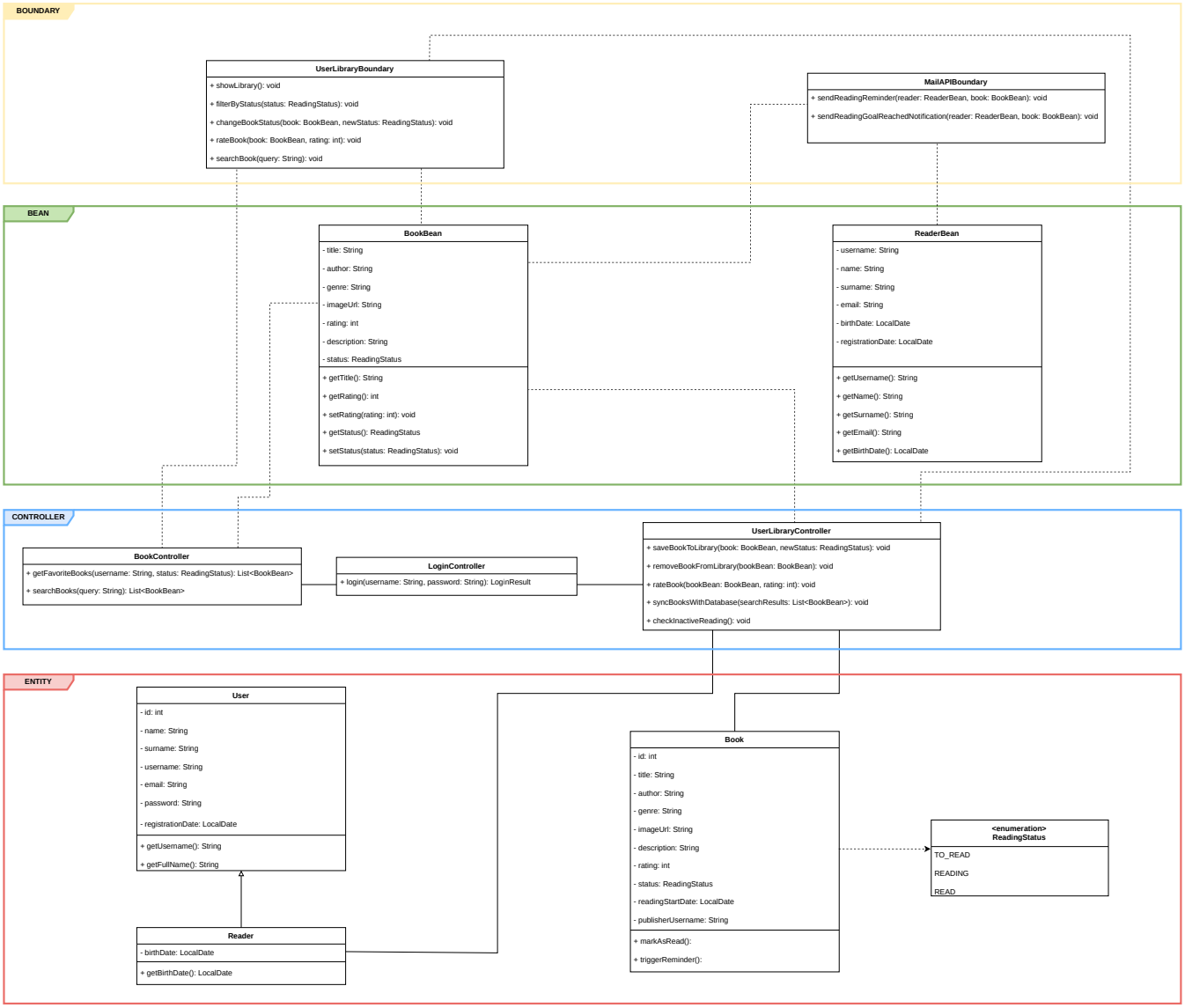

## Publisher Catalog



# 3. Design

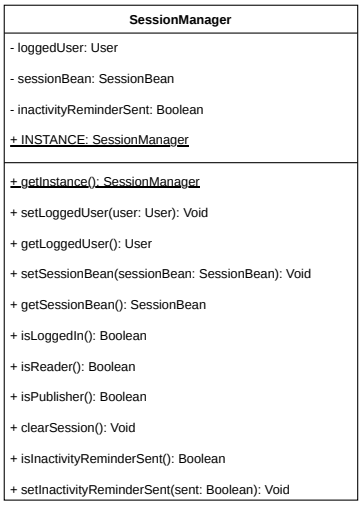
## 3.1 Class Diagram

### 3.1.1 VOPC analysis

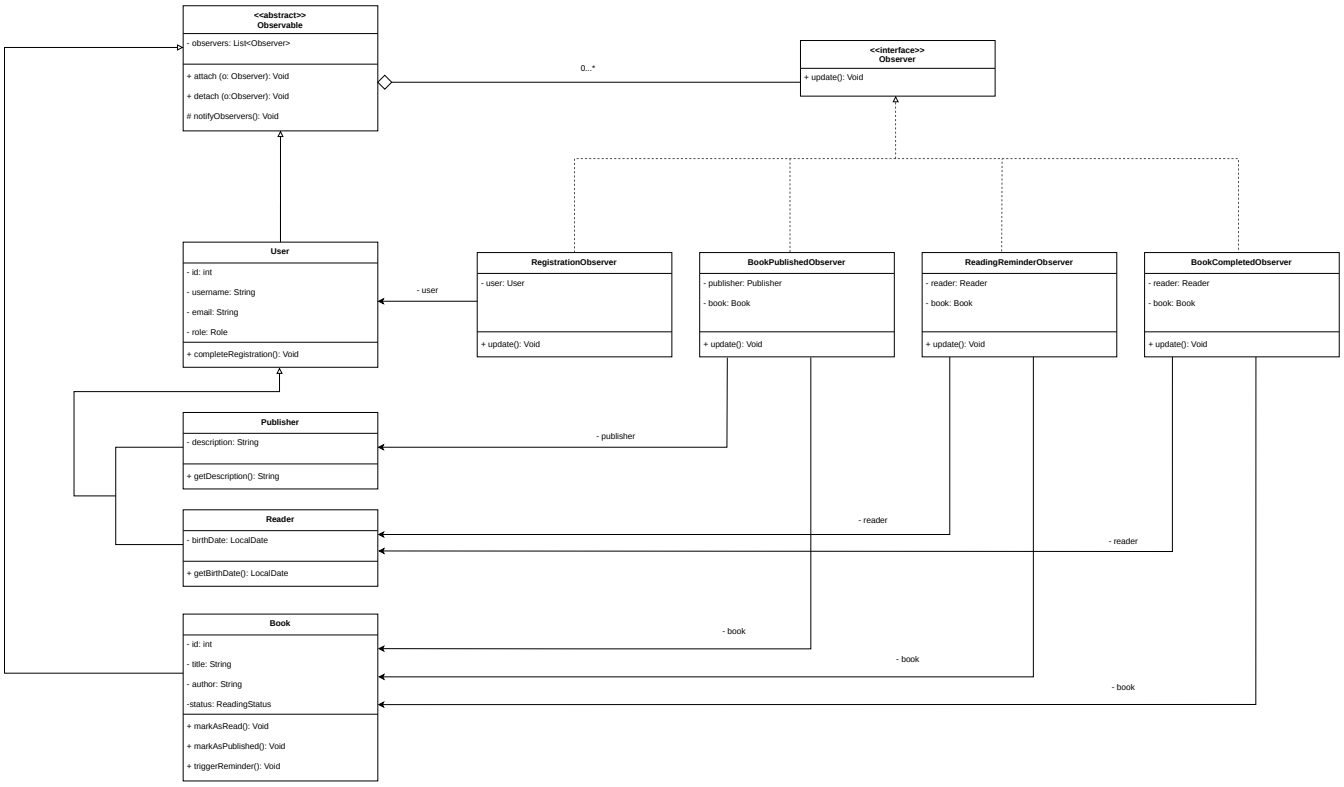


## 3.1.2 Design-level Diagram

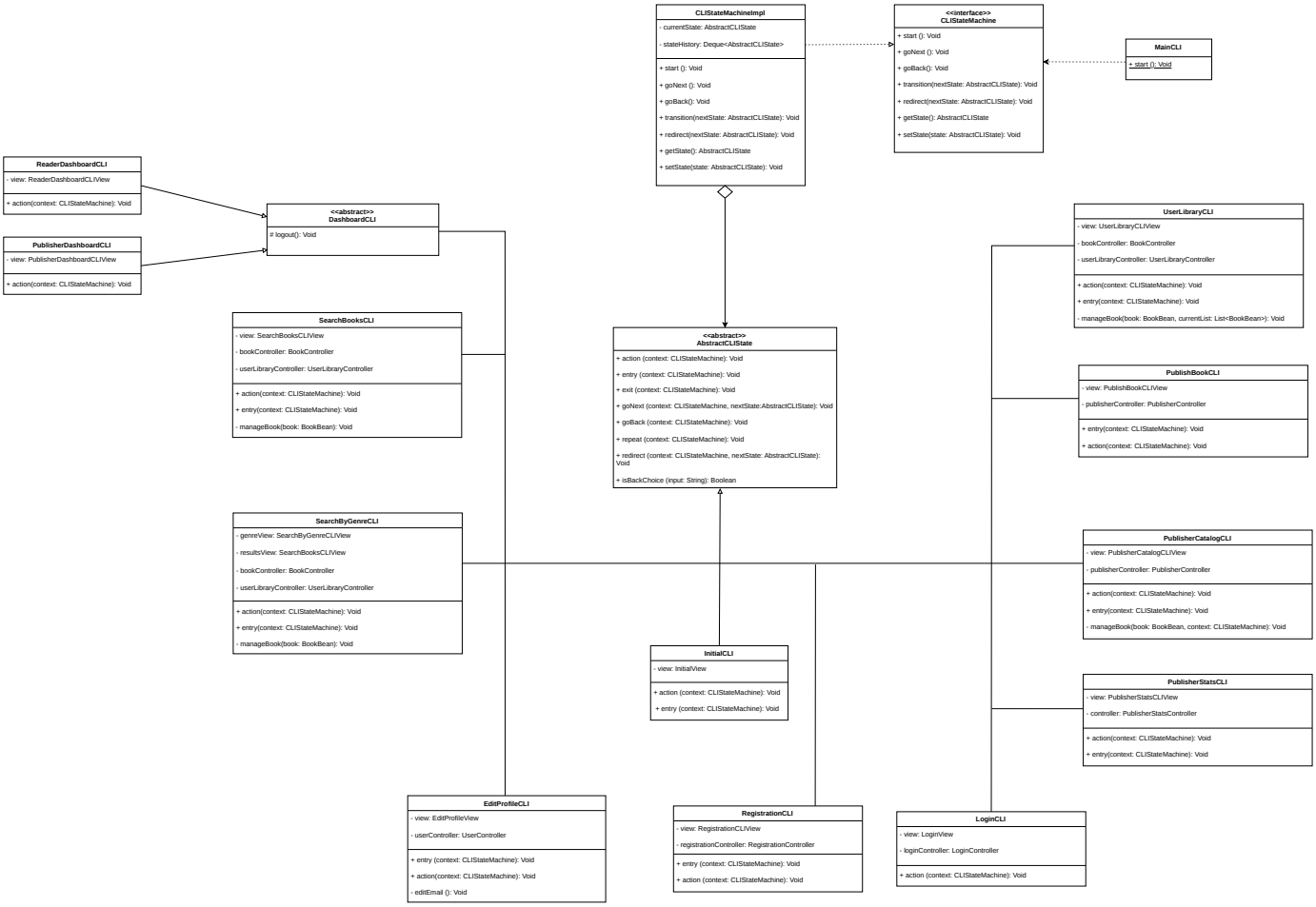
### Singleton Pattern:




### Observer Pattern:



State Pattern:



## 3.2 Design Patterns

### *1. SINGLETON*

I implemented the Singleton pattern for SessionManager to establish a single, shared source of truth for session data (such as the authenticated user and their role). This approach eliminates the need to manually pass session objects across controllers, while ensuring that exactly one instance exists throughout the application's lifecycle.
The implementation follows the "Initialization-on-demand holder" idiom. By utilizing a private static inner class, I achieved lazy initialization and inherent thread safety without the performance overhead of explicit synchronized blocks. Furthermore, SessionManager centralizes authorization logic; by exposing dedicated role-checking methods (isLoggedIn(), isReader(), isPublisher()), it allows controllers to verify permissions without requiring direct access to the User object.

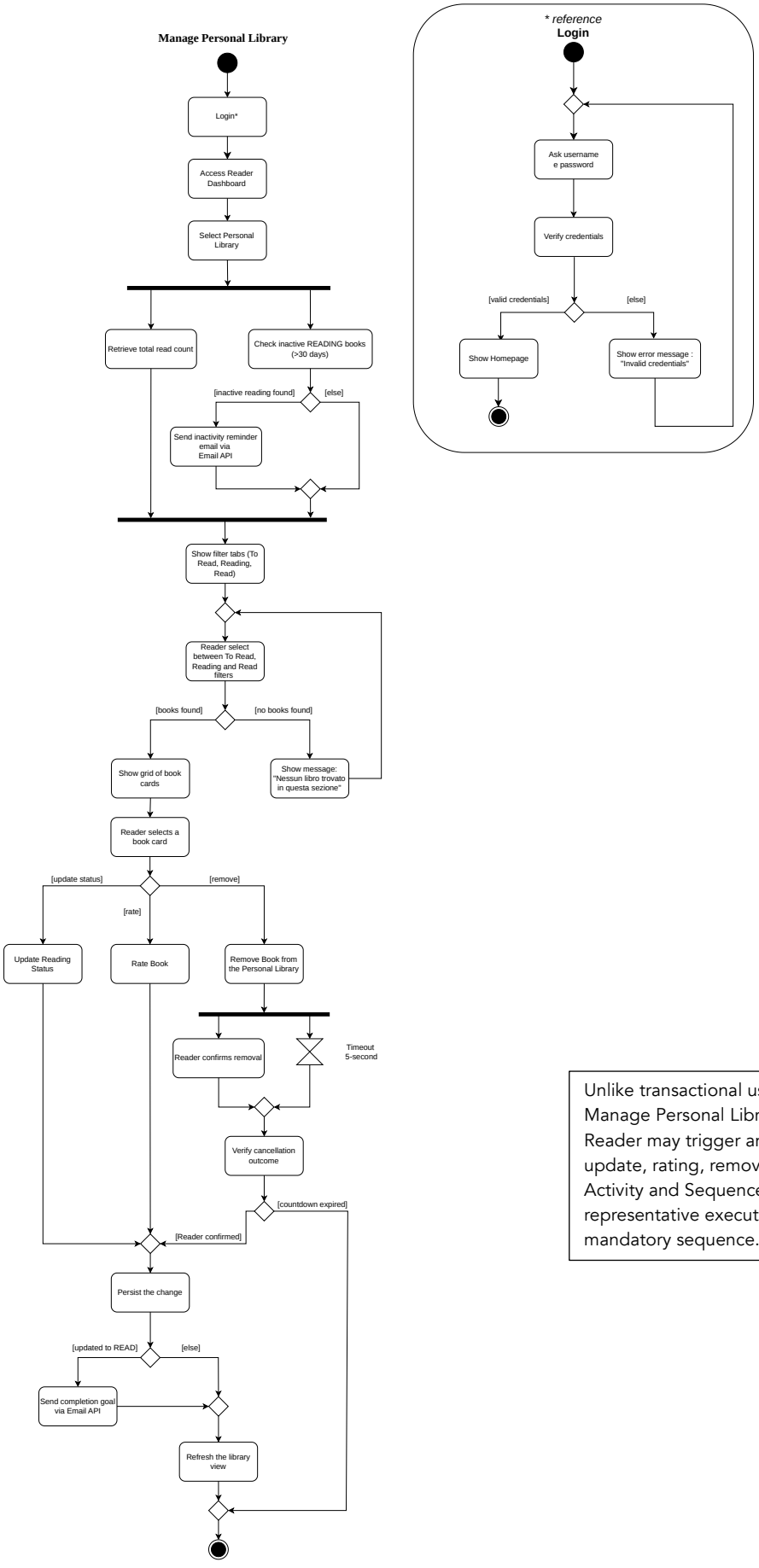
### *2. OBSERVER*

To manage notifications regarding reading progress, publication, and registration events, I adopted the Observer pattern. This design effectively decouples the Book and User entities, together with their respective controllers, from the NotificationService.
In this setup, the corresponding controller attaches the appropriate observer immediately before triggering the state-changing action — markAsRead(), triggerReminder(), markAsPublished(), or completeRegistration() — and detaches it right afterward. Upon the state change, the concrete observers BookCompletedObserver, BookPublishedObserver, ReadingReminderObserver, and RegistrationObserver automatically react to notify the reader or publisher by email. This architecture is highly extensible: should the need arise to incorporate new communication channels (e.g., SMS or push notifications) or new triggering events, we can simply introduce new observers without modifying existing business logic.

### *3. STATE*

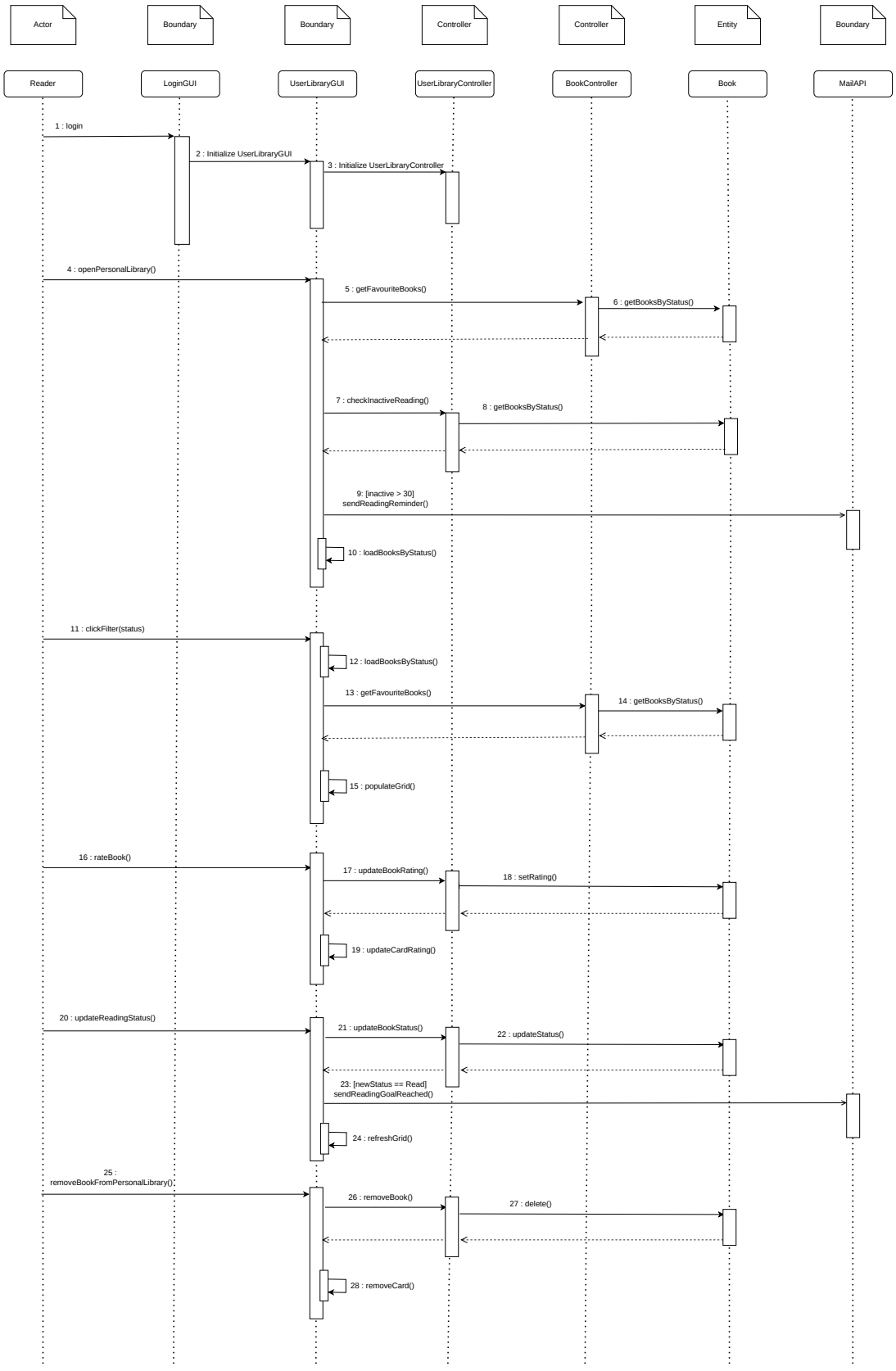
To avoid a rigid switch-case-based navigation logic in MainCLI, I implemented the State pattern.By transitioning to an object-oriented approach, I encapsulated each screen into a dedicated class extending AbstractCLIState. Each state manages its own lifecycle through three primary methods: entry() for initialization, action() for handling inputs and transition logic, and exit() for cleanup. MainCLI now simply instantiates CLIStateMachineImpl and invokes start(), delegating all screen management to the state machine itself, which acts as the context, maintaining the current state and a history stack (Deque<AbstractCLIState>) to enable seamless forward and backward navigation. Consequently, every screen is now a self-contained, autonomous module, making the system easy to extend while strictly adhering to the Open-Closed principle. For the two dashboard screens (ReaderDashboardCLI and PublisherDashboardCLI), I introduced an intermediate abstract class, DashboardCLI, extending AbstractCLIState, to centralize the shared logout logic and avoid code duplication between them.

# 3.3 Activity Diagram



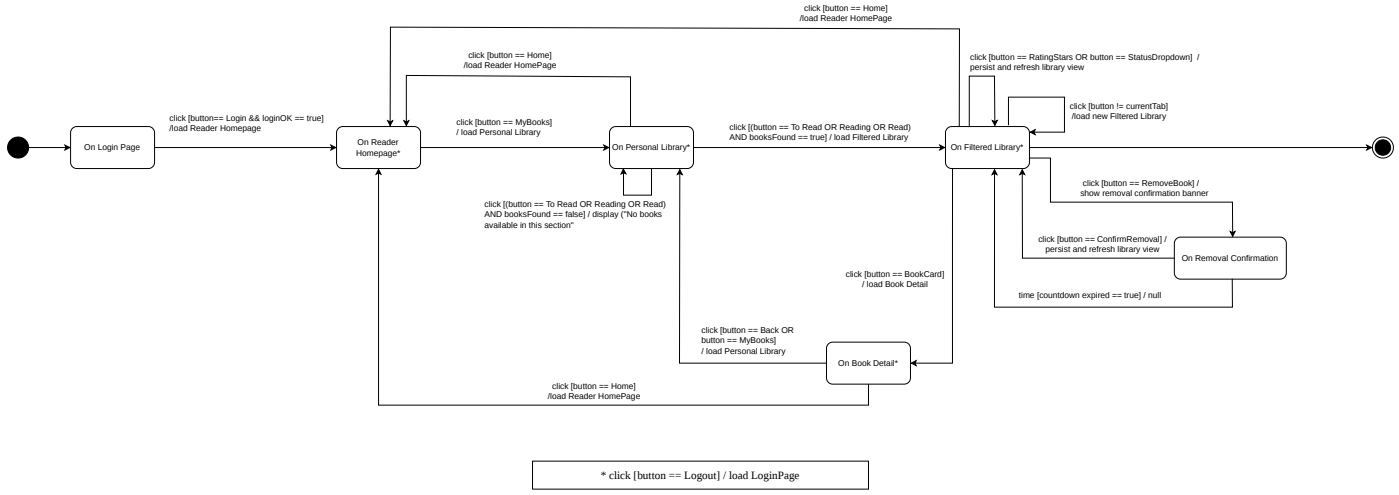

Unlike transactional use cases with a single deterministic path, Manage Personal Library is a management-style use case: the Reader may trigger any of several independent actions (status update, rating, removal) in any order and repeatedly. The Activity and Sequence Diagrams below illustrate one representative execution path exercising all branches, not a mandatory sequence.

# 3.4 Sequence Diagram




"Note: The Reader interactions (rating, updating status, and removing a book) are modeled sequentially to illustrate a standard use-case scenario of independent events triggered over time."

# 3.5 State Diagram



# 4. Testing

*Login with invalid credentials*

The authentication system was tested by attempting a login with an unregistered user or email address. This test ensures that the system correctly rejects invalid credentials by raising a LoginException. An additional test verifies that login attempts with an empty password are correctly rejected.

*Rejection of duplicate registrations*

The system was verified to prevent two accounts from sharing the same username or email address. After a first successful registration, a second attempt using the same username, or the same email address, correctly raises a RegistrationException.

*Saving and updating the user library*

The reader's library management was tested in-memory by registering a real Reader instance as the logged-in user through SessionManager. The first test verifies that saving a book with a specific reading status makes it appear correctly in the corresponding library section. The second test verifies that saving the same book with a new status removes it from the previous section and moves it to the new one, confirming that the update behaves as expected.

# 5. Code

GitHub:

[https://github.com/auroracicchetta/FindYourBook]

Sonar Qube Cloud:

[https://sonarcloud.io/project/overview?id=auroracicchetta_FindYourBook]

# 6. Video

[https://youtu.be/Hi_BrdKDSmU]